

Background

Please use the source code provided in this repository as your starting point:

`<https://github.com/danielhsul/sample-code.git>`

Inside, you will find a simple Golang web server, its unit tests, and two Dockerfiles (one for production build, one for the testing/linting environment). Your task is to design and implement an automated Continuous Integration (CI) pipeline that triggers automatically upon code commits.

Architectural Constraints & Considerations

To ensure an enterprise-grade and production-ready solution, your design must account for the following real-world constraints:

1. **Infrastructure Limitation:** Our team currently operates with a limited CI infrastructure consisting of **exactly 5 Build Servers (Nodes/Runners)**. During peak hours, it is highly likely that a single server will be triggered to run **3 or more CI build jobs concurrently**.
2. **Future Scalability (Multi-Repo):** While this assignment focuses on a single application, our company is adopting a microservices architecture. Your CI design must consider how this pipeline strategy can be easily scaled, maintained, and reused across **multiple different source repositories** in the future.

Requirements

You may use either **Jenkins** (`Jenkinsfile`) or **GitHub Actions** (`.github/workflows`) to complete this assignment. The pipeline must automate the following steps:

1. **Linting & Testing:** Execute code linting (`golangci-lint`) and unit tests using the provided `Dockerfile.test`.
2. **Build Docker Image:** Upon successful testing, build the application using the provided multi-stage `Dockerfile`.

Deliverables

Please submit your work via a GitHub repository link containing the following:

1. **The CI Pipeline Code** (`Jenkinsfile` or YAML file).
2. **A README.md** file introducing your CI pipeline design.

Your `README.md` **MUST** include:

- **A CI Pipeline Diagram or Flowchart:** This diagram must illustrate the entire flow starting from the **initial repository configurations** (e.g., branch protection rules, webhook settings, trigger events) down to the final image push.
- **An explanation of your design decisions**, specifically addressing:
 - **Resource Contention:** How your pipeline handles the limited nodes and concurrent build constraint.
 - **Scalability:** How your pipeline design can be scaled, maintained, and reused as we onboard multiple different source repositories.
 - **Tagging Strategy:** Your approach to Docker image versioning.
 - **Security:** How you securely manage and inject cloud credentials for pushing to the registry.